Homework 2 Basic Video Processing

This report presents image filtering, direct array-based shape drawing, consecutive-frame subtraction, and a nine-level Haar wavelet transform with exact reconstruction. All image results are generated by the included Python source.

Author: Anirudh Amuda. All four computational tasks have been executed and the generated results verified.

Human collaborators: none reported. Assistance: OpenAI ChatGPT assisted with implementation, explanations, and verification.

Problem 1 Convolution using kernels

Input: the original 512 x 512 grayscale Lena image extracted from the supplied assignment document. Computation uses float64, so negative derivative values and fractional filter responses are preserved. The output remains 512 x 512. Outside-image samples are zero, as requested.

The convolve function implements true convolution by reversing the kernel in both axes. For an even-sized kernel its anchor is floor(size/2); this agrees with same-sized mathematical convolution. Median filtering is nonlinear, so it uses a neighborhood median rather than a convolution sum.

Smoothing images retain their original intensity scale. Signed derivatives, gradient magnitudes, Laplacian, and DoG are displayed using min-max normalization: display = 255*(value-min)/(max-min). A constant response maps to zero. Raw float64 arrays are also saved as .npy files; display normalization never changes the calculations.

Exact Gaussian kernels

3 x 3: [[1,2,1], [2,4,2], [1,2,1]] / 16.

5 x 5: [[1,4,7,4,1], [4,16,26,16,4], [7,26,41,26,7], [4,16,26,16,4], [1,4,7,4,1]] / 273.
This is the matrix in the supplied figure.

Zero padding causes visible border darkening in the smoothing outputs and border responses in derivative outputs. These are expected consequences of the specified boundary rule, rather than scene edges.

Problem 1 Smoothing and first derivatives

Original



1 Median 3 x 3



2 Gaussian 3 x 3



3 Gaussian 5 x 5



4 Roberts magnitude



5 Prewitt magnitude



1. Median: replaces each pixel with the median of nine neighborhood samples. It suppresses isolated outliers while preserving many sharp boundaries better than averaging; small image details can still disappear.

2. Gaussian 3 x 3: a weighted local average that reduces high-frequency variation. Hair and feather texture become slightly softer, while major boundaries remain visible.

3. Gaussian 5 x 5: averages over a larger neighborhood and produces stronger smoothing than the 3 x 3 kernel. Fine texture is reduced further.

4. Roberts: uses exactly the assigned [-1,1] and vertical [-1;1] kernels, even though the classic Roberts cross operator uses diagonal 2 x 2 kernels. Adjacent-pixel differences reveal edges and remain sensitive to fine texture.

5. Prewitt: the assigned x and y kernels average across the perpendicular direction while differentiating in the other. The magnitude combines both directions using sqrt(gx*gx+gy*gy).

Problem 1 Sobel Laplacian and DoG

6 Sobel magnitude



7 Laplacian



8 Gaussian5 - Gaussian3



6. Sobel: uses [-1,0,1; -2,0,2; -1,0,1] and its perpendicular counterpart. The greater weight on the central row or column adds smoothing compared with Prewitt. Major contours are strong, although texture and padding borders also respond.

7. Laplacian: [0,1,0; 1,-4,1; 0,1,0] adds the horizontal and vertical second differences. The horizontal term is (I[x+1]-I[x])-(I[x]-I[x-1]); the vertical term has the same form. Thus it is literally differences of first differences. It is zero on constant interior regions, responds to rapid intensity curvature, and amplifies noise.

8. DoG: the 5 x 5 Gaussian output minus the 3 x 3 Gaussian output is computed before any rounding. Its signed range is then normalized to [0,255]. The result emphasizes boundaries and some fine-scale texture while suppressing the common low-frequency image content.

A difference of two Gaussian smoothings approximates a scaled Laplacian of Gaussian when their scales are related. In the continuous model, changing Gaussian scale is proportional to its spatial Laplacian. These small discrete kernels therefore give a qualitative approximation, not an exact LoG. The requested broader-minus-narrower order sets the sign of the response.

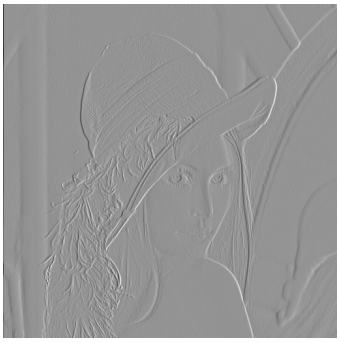
Derivative outputs and implementation

Both directional responses and their combined magnitude are saved for Roberts, Prewitt, and Sobel. Opposite signs correspond to opposite intensity transitions; convolution reverses the sign relative to applying the same antisymmetric kernel as correlation. Magnitude is unchanged by this sign reversal.

Functions used: numpy.pad, numpy.lib.stride_tricks.sliding_window_view, numpy.einsum, numpy.median, numpy.hypot, numpy.min/max, numpy.rint, numpy.clip, numpy.save, PIL.Image.open/fromarray/save. The actual kernels and custom convolve/normalize functions are in homework2.py.

Problem 1 Directional edge responses

Roberts x



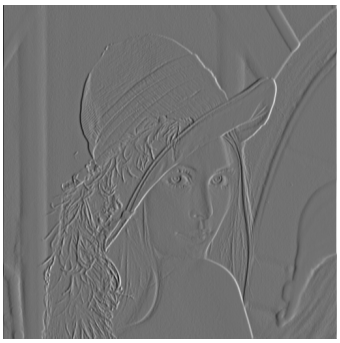
Roberts y



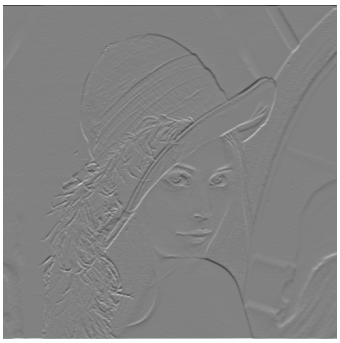
Roberts magnitude



Prewitt x



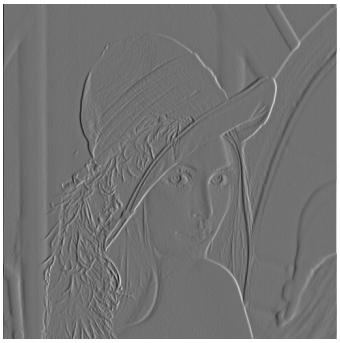
Prewitt y



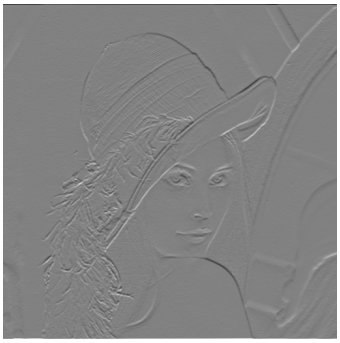
Prewitt magnitude



Sobel x



Sobel y



Sobel magnitude

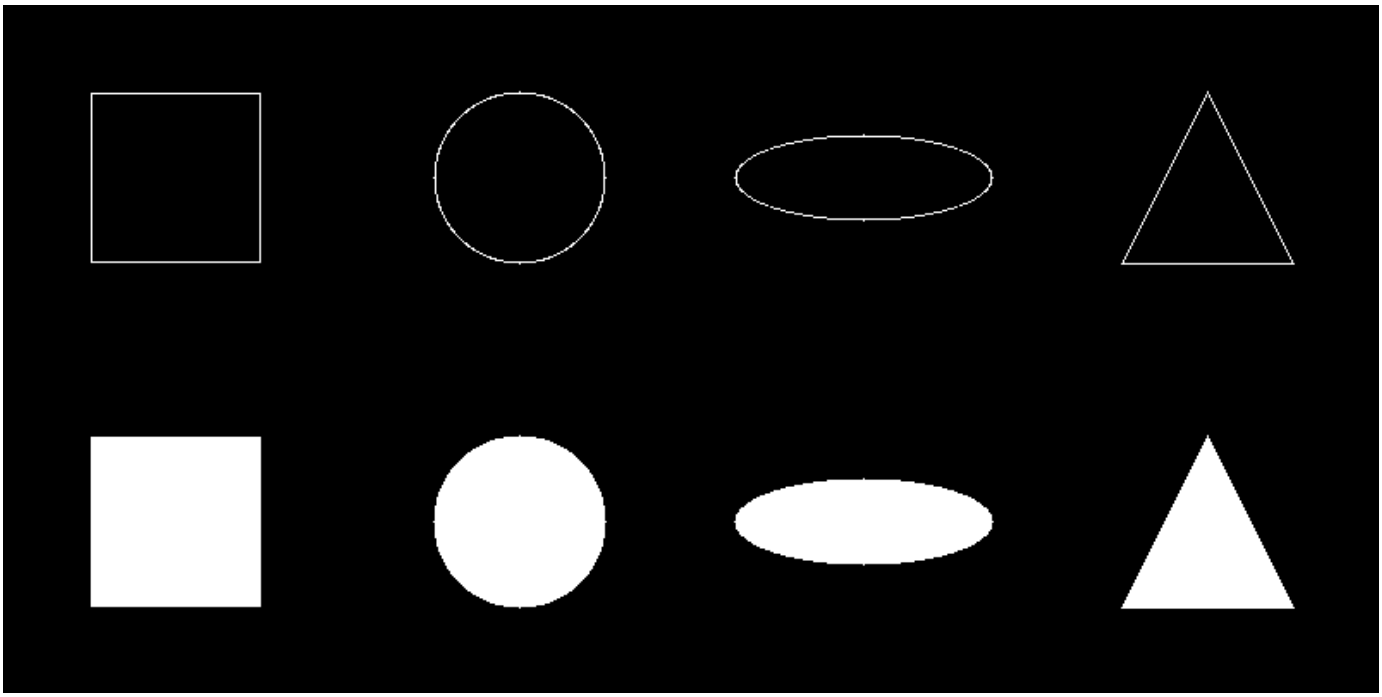


Each row shows the signed x response, signed y response, and combined gradient magnitude. Every panel is independently min-max normalized for visualization. Neutral interior signed responses appear near midgray, while unsigned magnitudes have a dark background. The raw arrays retain the actual scales for reproducibility.

For the x response, differences across columns emphasize vertical scene boundaries. For the y response, differences across rows emphasize horizontal boundaries. Magnitude combines both directions and highlights contours independently of transition sign.

Problem 2 Draw shapes

The generated image follows the reference: square, circle, ellipse, and triangle from left to right; outlined versions above and filled versions below. It uses an 800 x 400 uint8 canvas with black background (0) and white shapes (255). Each square, circle, and triangle spans about 100 x 100 pixels; the ellipse has a 150 x 50 bounding box to match the elongated reference.



The implementation constructs coordinate grids with numpy.indices and writes shape masks directly into the array. Squares use coordinate bounds. Circles and ellipses use ((x-cx)/rx)^2 + ((y-cy)/ry)^2 <= 1. Triangles use bounds and linear edge inequalities. Outlines subtract an inner region from the filled mask; the triangle outline removes pixels whose four adjacent neighbors are also inside.

No prebuilt shape-drawing function is used. NumPy boolean indexing assigns all selected pixels at once. Slice endpoints follow Python convention: the upper bound is excluded, so a range that must include a last index requires last+1.

Functions used: numpy.zeros, numpy.indices, absolute-value operations, boolean masks, NumPy slicing and indexing, and PIL.Image.fromarray/save. Custom function: problem2.

Problem 3 Video and frame subtraction

Measured result: 120 input frames and 119 difference frames, at 24 fps and 320 x 240. Both FFV1 outputs were decoded and verified pixel-for-pixel against their RGB PNG source sequences.

difference_msmpeg4v2.avi: 499,748 bytes.

original_ffv1.avi: 4,068,792 bytes.

original_msmpeg4v2.avi: 578,072 bytes.

difference_ffv1.avi: 3,588,626 bytes.

Input: the supplied phone recording in data/input.mp4. It is 6.57 seconds long, contains H.264 video at nominal 30 fps and AAC audio, and has stored dimensions 1024 x 576. FFmpeg honors the orientation metadata; the displayed view is portrait. Only its first five seconds are processed.

The Python pipeline calls FFmpeg to decode the first five seconds, preserve the aspect ratio while fitting within 320 x 240, pad to that canvas size, resample to 24 fps, and save RGB PNG frames. Resampling standardizes playback timing, rather than asserting that the original camera captured at 24 fps.

The same canonical PNG sequence is encoded into original_msmpeg4v2.avi and original_ffv1.avi. The lossy encoder uses q:v=2 and yuv420p; the lossless encoder uses bgr0 to preserve RGB samples exactly. PNG frames are used for subtraction to avoid introducing lossy re-encoding noise before the calculation.

For each successive pair, D[n] = clip(int16(I[n+1]) - int16(I[n]) + 128, 0, 255). Signed conversion before subtraction prevents uint8 wraparound. A pixel that does not change maps to 128; brighter changes map above 128 and darker changes below 128. Differences outside [-128,127] saturate as required.

With N decoded images, there are N-1 differences. The handout example lists img00 through img99 (100 files) but also says 99 images; its stated 98 output frames applies only to 99 actual input files. The implementation uses the actual decoded count.

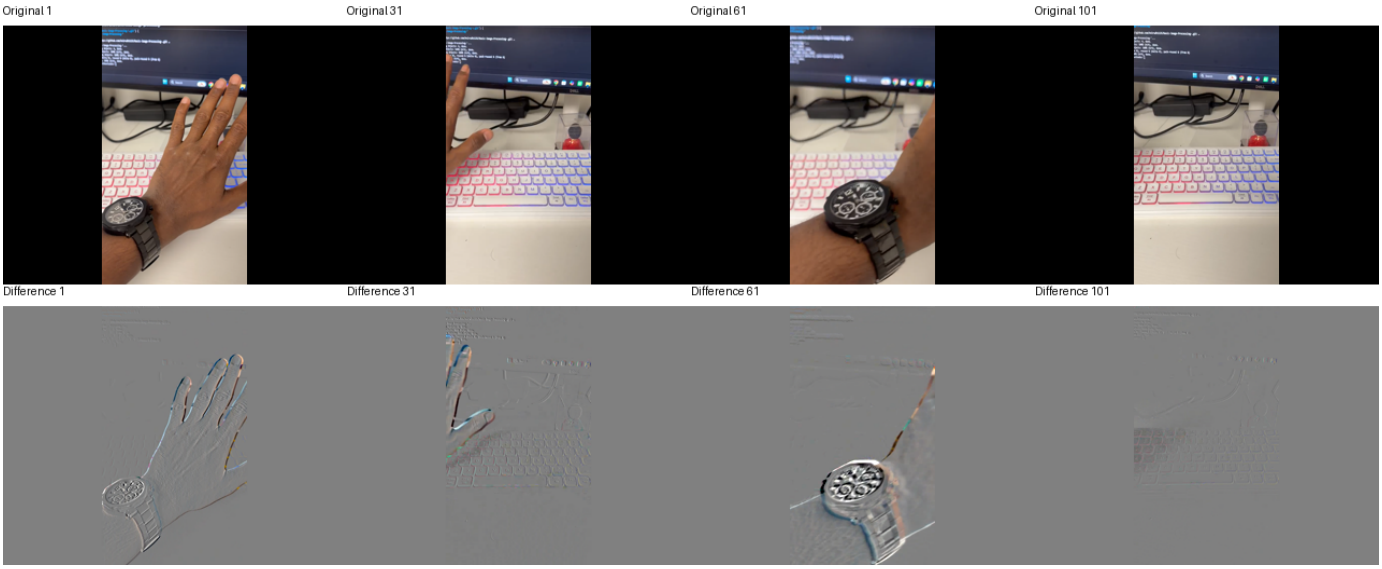
The difference sequence is encoded into difference_msmpeg4v2.avi and difference_ffv1.avi at 24 fps. Audio is excluded. FFV1 preserves the already shifted and clipped difference frames; it cannot reverse information lost by the mandated clipping operation.

The recording shows a hand and wristwatch moving in front of a keyboard and monitor. Hand and watch boundaries are visible as bright and dark differences. The camera background also shifts slightly, so keyboard and monitor edges respond; this is not a perfectly stationary capture. The method still reveals motion, but camera jitter and video noise prevent a clean isolated foreground blob.

Functions used: pathlib.Path, subprocess.run, PIL.Image.open/convert, numpy.array/astype/clip, and custom frame_difference/problem3. FFmpeg performs decoding, scaling, frame-rate conversion and AVI encoding.

Problem 3 Measured video results

The first five seconds produce 120 RGB PNG frames at 24 fps. Consecutive subtraction produces 119 frames and a 119/24 = 4.9583 second difference video. The images below show matched samples from the input sequence and the difference sequence.



Original sequence samples appear above; shifted and clipped differences appear below. Unchanged pixels approach neutral gray (128); the black padding of the original frames also becomes gray in the difference images because its temporal difference is zero.

The moving hand and watch give clear high-contrast contours. Around frame 101 the hand has left the view; residual background edges indicate slight camera movement and imaging variation. No denoising, stabilization, or thresholding is applied because this task requests the raw shifted frame differences.

Original AVI: msmpeg4v2 578,072 bytes; FFV1 4,068,792 bytes (about 7.04 times larger). Difference AVI: msmpeg4v2 499,748 bytes; FFV1 3,588,626 bytes (about 7.18 times larger). The lossy files are smaller here; the comparison includes AVI container overhead and depends on these encoder settings.

Both FFV1 videos were decoded back to RGB and matched every corresponding PNG pixel exactly. This verifies losslessness relative to the processed frames, not relative to the phone sensor: the uploaded phone video is already compressed, and resampling/scaling precede encoding.

Required files: original_msmpeg4v2.avi, original_ffv1.avi, difference_msmpeg4v2.avi, and difference_ffv1.avi, all in results/problem3. Input and difference PNG sequences are included for reproducibility.

Problem 4 Haar wavelet transform

Each 2 x 2 block is represented by average A, horizontal H, vertical V, and diagonal D, using the exact normalization in the handout:

A = (I00 + I01 + I10 + I11) / 4

H = (I00 + I01 - I10 - I11) / 4

V = (I00 - I01 + I10 - I11) / 4

D = (I00 - I01 - I10 + I11) / 4

The image layout is [[A,H],[V,D]], following the reference diagram. Only A is transformed at the next level. The sizes of A after levels 0 through 8 are 256, 128, 64, 32, 16, 8, 4, 2, and 1 pixels on each side.

All coefficients are float64. The three detail arrays at every level are retained in a Python list and saved in coefficients.npz; only the final 1 x 1 average is needed for reconstruction. Displayed details are shifted by 128 and clipped/rounded to uint8. Those display images are never used for reconstruction.

Each saved layer image is a full-size 512 x 512 recursive mosaic. Previously produced detail bands remain in place as only the top-left average region is replaced by the next decomposition. Fine-scale differences are mostly near midgray, while strong structure survives into coarser layers.

Level 0



Level 1



Level 2



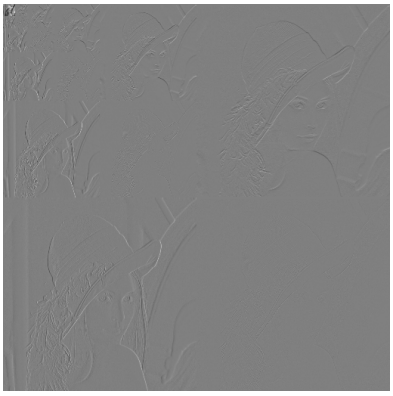
Functions used: NumPy stride slicing, elementwise arithmetic, numpy.empty, numpy.savez, a Python list, PIL.Image.fromarray/save, and custom haar_layer/problem4.

Problem 4 Remaining transform layers

Level 3



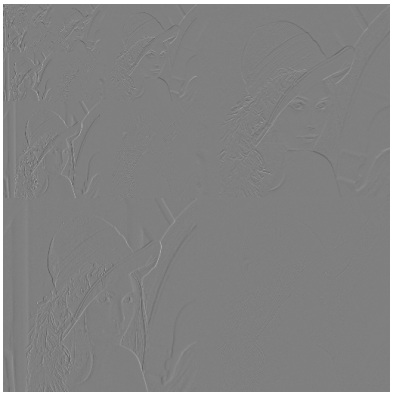
Level 4



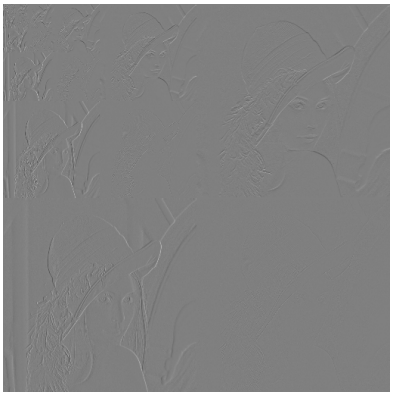
Level 5



Level 6



Level 7



Level 8



The shrinking average block summarizes increasingly large regions. Detail bands retain contrasts at their respective scales. The final average is 123.607799530029, equal to the mean intensity of the original image.

The decomposition rearranges image information into scale and orientation bands; by itself it does not reduce the number of coefficients. Many small detail coefficients can be coded efficiently, but discarding or quantizing them changes the reconstruction.

Problem 4 Exact reconstruction

I00 = A + H + V + D

I01 = A + H - V - D

I10 = A - H + V - D

I11 = A - H - V + D

Reconstruction starts from Apix and consumes the saved detail triples in reverse level order, 8 down to 0. Every inverse step doubles the dimensions: 1 to 2 to 4 to 8 to 16 to 32 to 64 to 128 to 256 to 512. The handout I02 is interpreted as I10, the lower-left pixel.

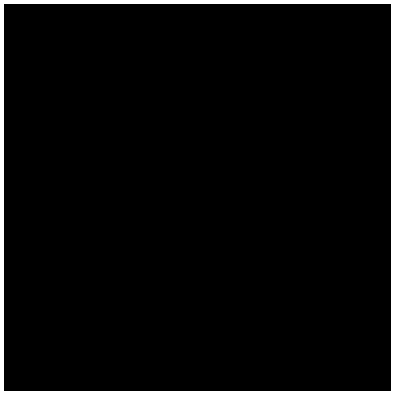
Original



Reconstructed



Absolute error



Measured maximum absolute reconstruction error: 0.0. Mean absolute error: 0.0. Exactly equal pixels: 262,144 of 262,144. Error is calculated on the float64 reconstruction before uint8 conversion. The included assert enforces zero error.

For this 8-bit input and nine stages, division by powers of two is represented exactly in float64 at the required precision. Therefore retaining every coefficient permits exact inversion. Rounding the coefficients to display images would destroy this guarantee.

Compression discussion

Haar concentrates smooth content in averages and local changes in details. This supports multiresolution representation and efficient coding when details are small. Wavelet coding can reduce the block-boundary artifacts associated with heavily quantized 8 x 8 DCT JPEG, although performance depends on the image and coding method. Lossless coding also requires reversible processing and retention of all necessary coefficient information.

Technical clarification: JPEG uses DCT, not DCE. JPEG2000 Part 1 uses reversible 5/3 or irreversible 9/7 wavelet filters, rather than the Haar filter implemented here. Haar demonstrates the multiscale principle; this experiment implements no JPEG2000 encoder and makes no measured compression-ratio claim.

Reference: ITU-T T.803 (2024), JPEG2000 conformance testing, and JPEG committee overview: https://jpeg.org/jpeg2000/ .

Verification: independent reference convolution comparisons, small-array Haar inversions, and signed-difference/clipping checks passed. The video pipeline was exercised using a separate synthetic test clip; that test is not submitted as a student camera recording.